SAPIENZA
UNIVERSITÀ DI ROMA

Computer Vision

DIAG
Dipartimento di Ingegneria informatica, automatica e gestionale Antonio Ruberti

Project 7: Lean Geometry [GitHub Link]

Boosting Mobile MDE with LeJEPA

Students: Franzoso Antonio (2133047), Liberti Paolo (1787999)

Professors: Amerini Irene, Teglia Simone

15/06/2026

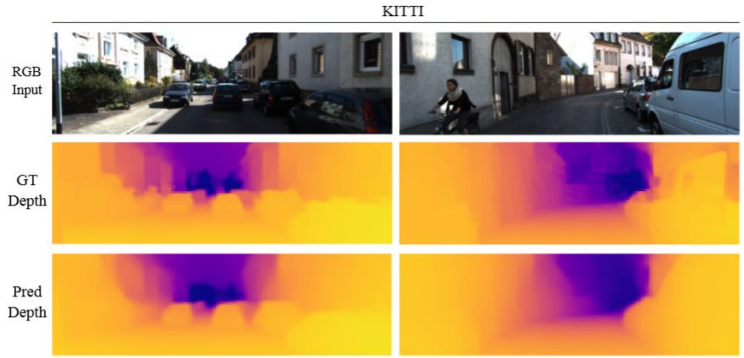
Outline

- *Problem Statement*
- *State of the Art in MDE*
- *Proposed Method*
- *Datasets*
- *Experimental Setup*
- *Model Evaluation*
  - *PCA*
  - *MDE Results*
- *Conclusions & Future Work*

Problem Statement

High-quality Monocular Depth Estimation (MDE) typically relies on heavy backbones and massive supervision.

1) Mobile MDE models (METER) start from random weights - slow convergence, fragile features
2) Supervised pre-training (ImageNet) is expensive and needs labels + hundreds of GPU-hours
3) Self-supervised methods rely on teacher-student tricks or masking heuristics



Focus: Direct comparison of the same METER architecture trained from scratch (baseline) vs. initialized with LeJEPA self-supervised pre-training

State of the Art in MDE

Classic (CNN-based)

Pros: Strong local feature extraction, excellent at capturing high-frequency details (edges, textures).
Cons: Limited receptive field leads to poor global context understanding. Deep CNNs remain unsuitable for real-time inference on low-resource hardware.

Accurate (ViT)

Pros: Leverage Self-Attention for strong global context and long-range dependency understanding.
Cons: Computationally expensive (heavy backbones). Unsuitable for real-time inference on edge devices. Often rely on massive supervised pre-training.

State of the Art in MDE

Lightweight

Pros: Designed for low latency, energy efficiency, and fast inference on embedded GPUs (e.g., NVIDIA Jetson).
Hybrid Approach (METER): Combines CNNs and lightweight Transformers to balance local details and global context without the overhead of heavy ViTs.
Cons: Often sacrifice fine detail and metric accuracy compared to heavyweight models if trained conventionally.

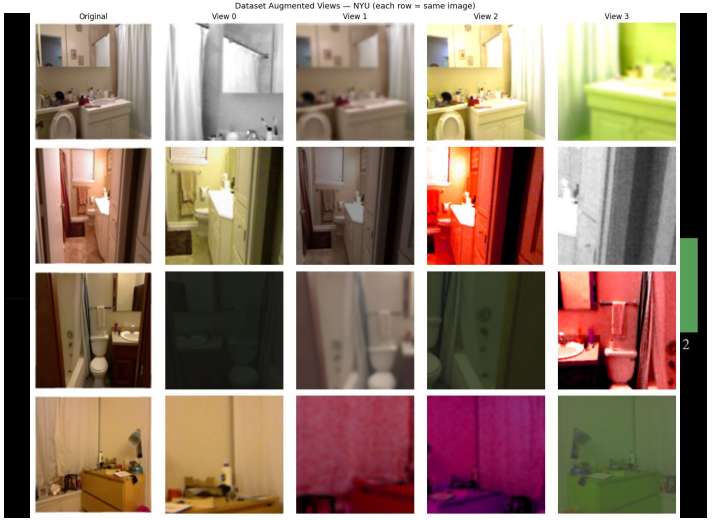
Learning (SSL)

The Problem: Traditional supervised training requires millions of expensive, pixel-perfect depth maps. Standard SSL methods (like DINO or I-JEPA) suffer from representation collapse and rely on complex heuristics (teacher-student, stop-gradients).
The Solution (LeJEPA): Provides a provable, heuristics-free objective (SIGReg) to train representations.
Impact on MDE: Allows lightweight models to learn dense, high-quality geometric features natively from raw, unlabeled images, boosting downstream depth accuracy.

Proposed Method - LeJEPA

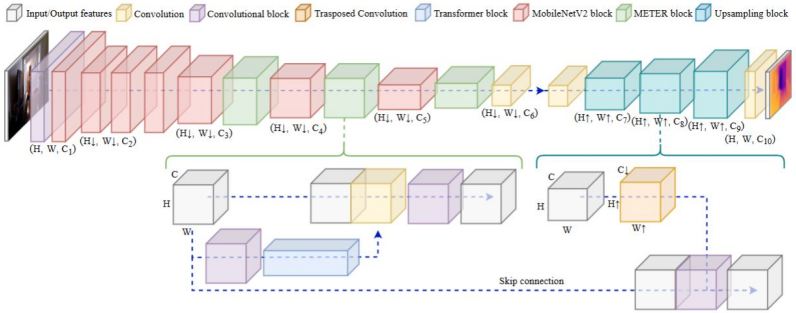
- **Self-Supervised Pre-training (LeJEPA)**
  - Joint Embedding Architecture that trains the METER encoder backbone to produce high-dimensional augmentation-invariant representations
    - one image → V augmented views → all map to almost the same point in embedding space (L_{inv})
    - It adds SIGReg to prevents representation collapse by forcing embeddings to be spread out (Gaussian-distributed) across the batch (L_{SIGREG})
- Adapted from [official minimal implementation] for ViT-like models

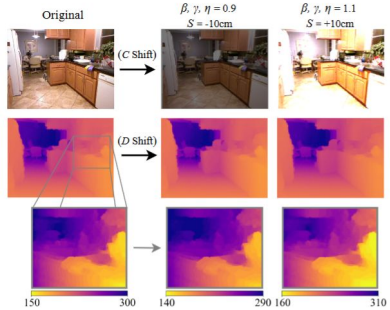
$$\mathcal{L} = (1 - \lambda) \cdot \mathcal{L}_{inv} + \lambda \cdot \mathcal{L}_{SIGReg}$$



Proposed Method - METER

- **Architecture**
  - We replicated the [METER encoder-decoder architecture] in PyTorch, supporting all three model variants (XXS: 0.71M, XS: 1.45M, S: 3.29M params)
  - The encoder is a hybrid CNN-Transformer backbone (MobileNetV2 inverted residual blocks + single-layer MobileViT attention blocks) that extracts multi-scale features
  - The decoder recovers full-resolution depth maps via transposed convolutions and skip connections
- Reproduced default augmentation policy (vertical flip, mirroring, random crop, channel swap) + [METER shifting strategy] (color shift, depth shift)
- Implemented the Balanced Loss Function (BLF)





$$L(y_i, \hat{y}_i) = L_{depth} + \lambda_1 L_{grad} + \lambda_2 L_{norm} + \lambda_3 L_{SSIM} \quad (1)$$

Datasets

- **NYU (indoor)** - 46028 train/654 test [link]
  - depth maps maximum distance of 10 meters
  - trained at 256x192

- **KITTI (outdoor)** - 22285 train/652 test [rgb link, depth maps link]
  - sparse depth maps maximum distance of 80 meters
  - trained at 640x192

B. Benchmark datasets

The datasets used to show the performance of METER are NYU Depth v2 [11] and KITTI [12], two popular MDE benchmark datasets for indoor and outdoor scenarios.

NYU Depth v2 dataset provides RGB images and corresponding depth maps in several indoor scenarios captured at a resolution of 640×480 pixels. The depth maps have a maximum distance of 10 meters. The dataset contains $120K$ training samples and 654 testing samples; we used for training the $50K$ subset as performed by previous works [5], [16]. The input images have been downsampled at a resolution of 256×192.

KITTI dataset provides stereo RGB images and corresponding 3D laser scans in several outdoor scenarios. The RGB images are captured at a resolution of 1241×376 pixels. The depth maps have a maximum distance of 80 meters. We train our network at a input resolution of 636×192 on Eigen et. al [13] split; it is composed of almost $23K$ training and 697 testing samples. Similarly to [21], due to the low density depth maps, we evaluate the compared models in the cropped area where point-cloud measurement are reported.

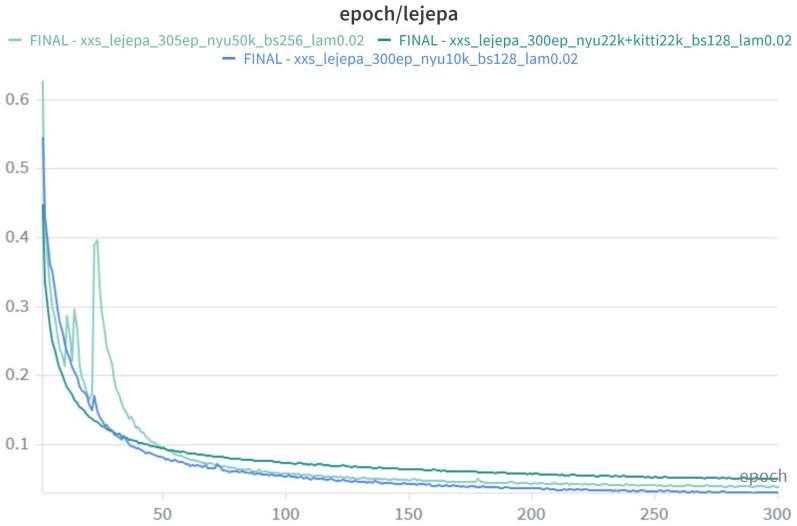
Experimental Setup

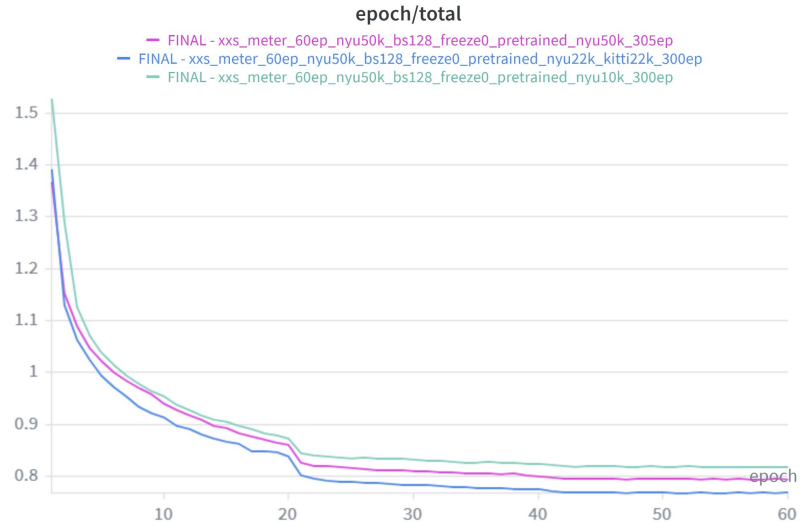
- **Experiment management**
  - Hydra (configuration)
  - Weights & Biases (cloud run tracking)
  - uv (dependency management)

- **OS:** Fedora Linux 43
- **RAM:** 32GB DDR5
- **CPU:** AMD Ryzen 7 7700 (16 core)
- **GPU:** NVIDIA GeForce RTX 5060 Ti

NAME (14 of 26 listed)	STATE	NOTES	RUNTIME
FINAL ...cratch	Finished	NYU 50k 60ep scratch XXS	44m 37s
FINAL ...305ep	Finished	NYU 50k 60ep LeJEPA 50k XX...	44m 10s
FINAL ...300ep	Finished	NYU 50k 60ep LeJEPA (mix 4...	44m 4s
FINAL ...300ep	Finished	NYU 50k 60ep LeJEPA 10k XXS	43m 48s
FINAL ...cratch	Finished	KITTI Scratch XXS	1h 13m 38s
FINAL ...300ep	Finished	KITTI Pretrained XXS	4h 56m 46s
FINAL ...300ep	Finished	KITTI 22k 60ep LeJEPA (KITT...	58m 12s
FINAL ...m0.02	Finished	Pretrain NYU 50K (if we have ...	11h 9m 54s
FINAL ...m0.02	Finished	Pretrain NYU+KITTY 44K (22...	10h 44m 5s
FINAL ...m0.02	Finished	Pretrain NYU 10K	2h 39m 29s
FINAL ...m0.02	Finished	Pretrain KITTI 22k	5h 42m 41s
FINAL ...cratch	Finished	NYU Scratch XS	1h 14m 22s
FINAL ...300ep	Finished	NYU Pretrained XS	1h 13m 4s
FINAL ...m0.02	Finished	Pretrain NYU+KITTY 44K (22...	11h 42m 21s

Experimental Setup - LeJEPA





After playing with different parameters, we decided to use as baseline **LeJEPA pretrained for 300ep with a mix of NYU and KITTI (22k/22k)** as we noticed that depth loss on the METER model pretrained with this was lower than the one pretrained just on a subset of NYU (10k), or on the full NYU only (50k).

Experimental Setup - METER

- We tried to replicate the exact same hyperparameters described in the original METER paper

- **MDE finetuning *(from scratch)***
  - 50k NYU RGB+depth samples, 60 epoch, batch size 128
  - 22k KITTI RGB+depth samples, 60 epoch, batch size 128

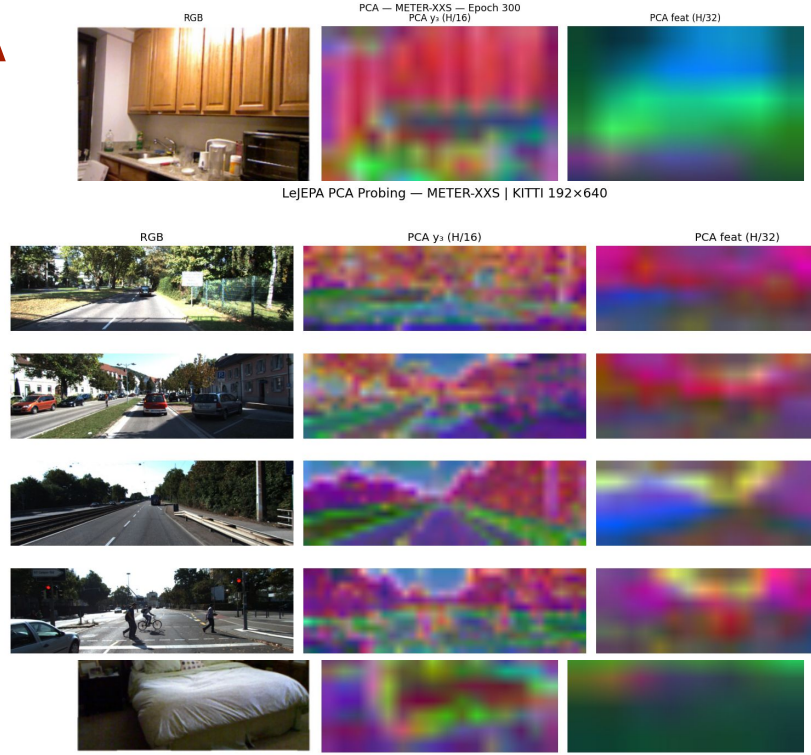
- **MDE finetuning *(with LeJEPA pretrained on NYU&KITTI encoder)***
  - 50k NYU RGB+Depth samples, 60 epoch, batch size 128
  - 22k KITTI RGB+depth samples, 60 epoch, batch size 128

A. Training hyper-parameters

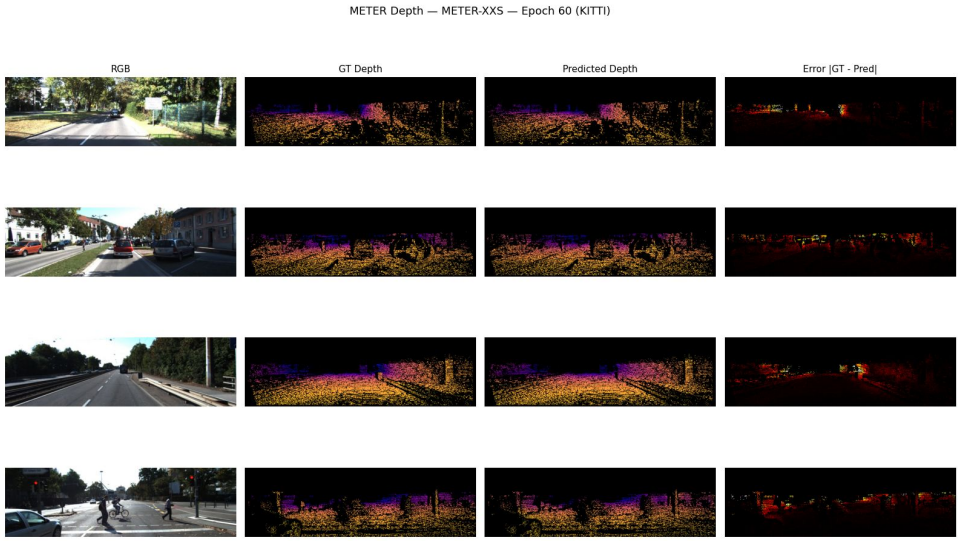
METER has been implemented using PyTorch[2] deep learning API, randomly initializing the weights of the architectures. All the models have been trained from scratch using the AdamW optimizer [42] with $\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay $wd = 0.01$ and an initial learning rate of 0.001 with a decrement of 0.1 every 20 epochs. We use a batch size of 128 for a total of 60 epochs. For the *balanced loss function* we empirically choose the scaling factors $\lambda_1 = 0.5$ and $\lambda_2, \lambda_3 = \{1, 10, 100\}$ depending on the unity of measure used for the predicted depth map, i.e. meters, decimeters or centimeters. We apply a probability of 0.5 for all the random transformations set in the data augmentation policy.

Model Evaluation - PCA

- Project spatial feature maps of the unlabeled, pre-trained encoder to the first 3 principal components (mapped to RGB).
- **Findings**: Latent space natively segments roads, sky, vehicles, walls, and furniture.
- **NYU (Indoor)**: Segregates foreground beds/sofas (warm colors) from walls and floors.
- **KITTI (Outdoor)**: Segregates roads (purple/green) from sky (pink) and trees/buildings (orange).
- **Conclusion**: The encoder even being low dimensional learns geometry before downstream depth labels are introduced.

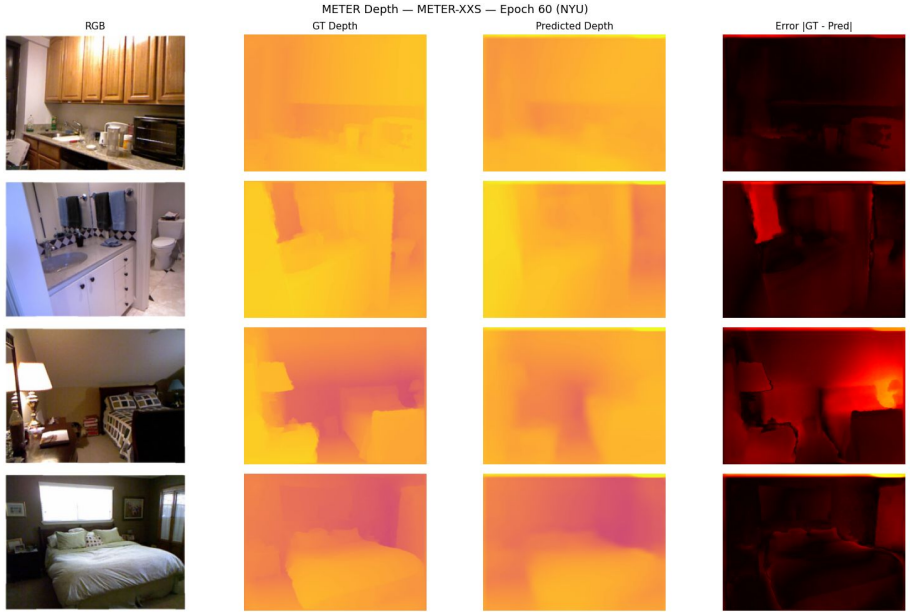


Model Evaluation - KITTI XXS

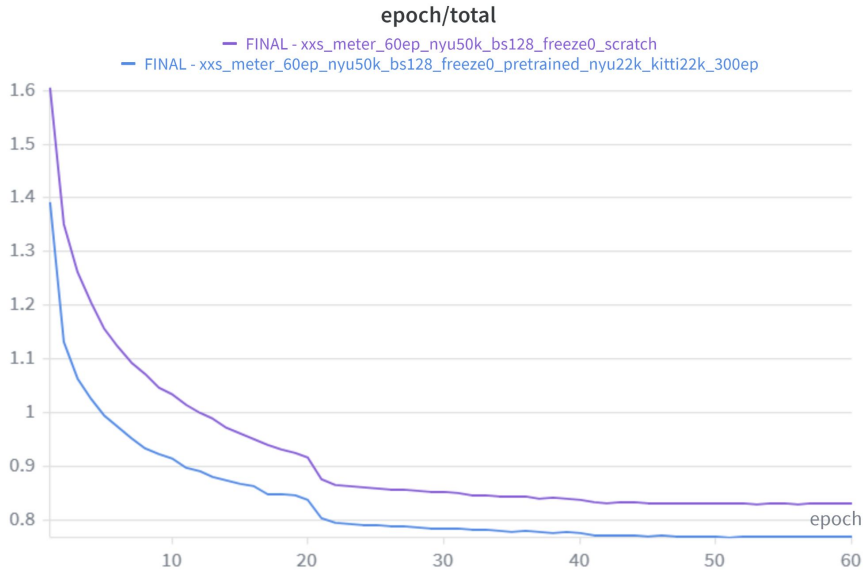


LeJEPA Scratch *LeJEPA Pretrained*

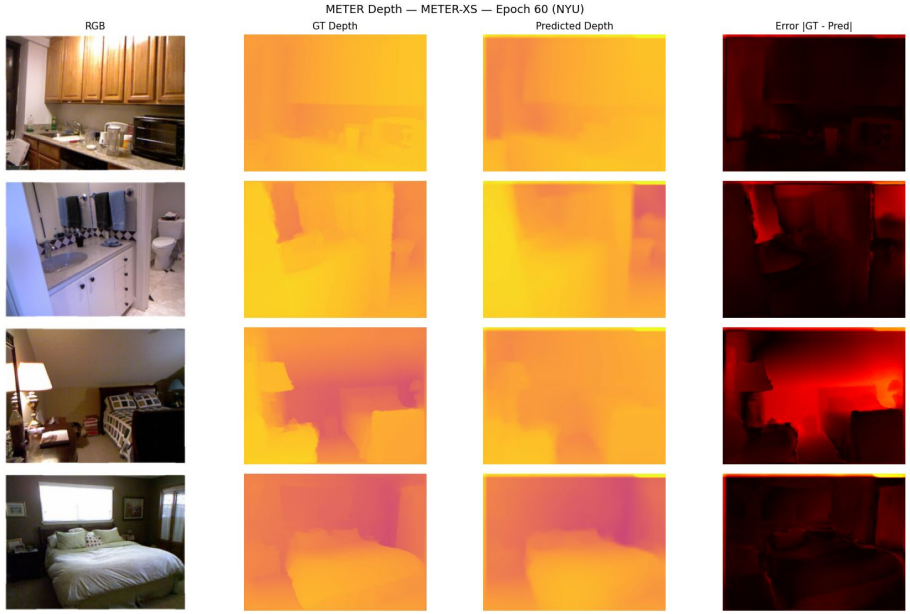
Model Evaluation - NYU XXS



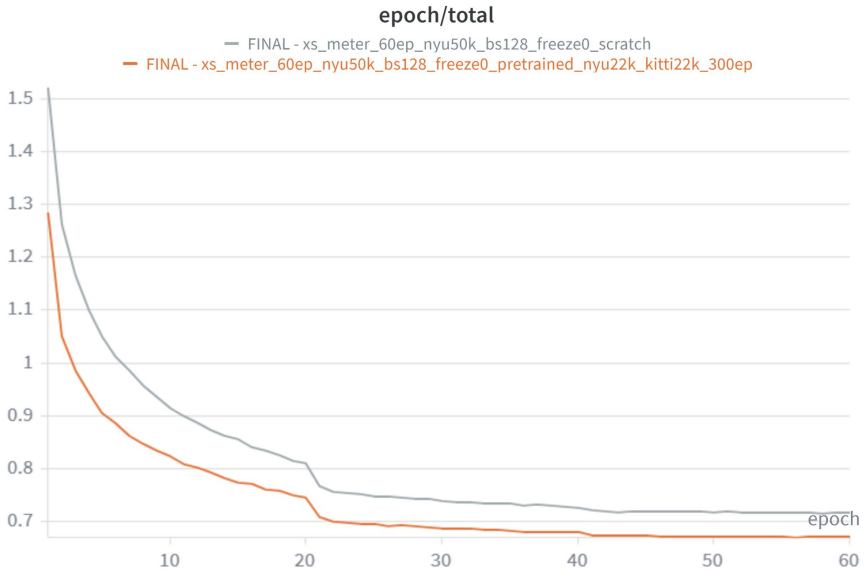
LeJEPA Scratch *Pretrained*



Model Evaluation - NYU XS

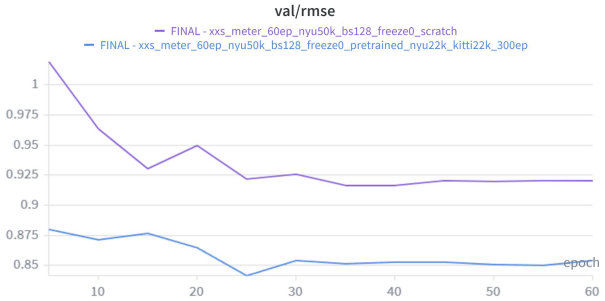


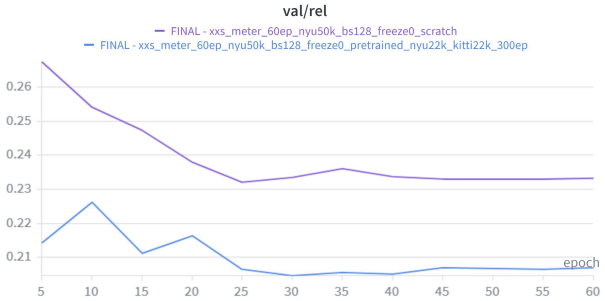
LeJEPA Scratch *Pretrained*

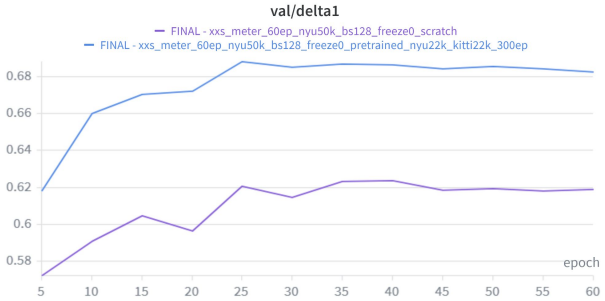


Metric	Supervised (Scratch)	LeJEPA Transfer (Ours)	Improvement
δ_1 (↑)	0.7932	**0.8325**	*+4.95%*
δ_2 (↑)	0.9381	**0.9587**	*+2.20%*
δ_3 (↑)	0.9803	**0.9895**	*+0.94%*
RMSE (↓)	5.4166	**4.9096**	*-9.36%*
AbsRel (↓)	0.1454	**0.1211**	*-16.71%*
log10 (↓)	0.0679	**0.0533**	*-21.50%*

Quantitative Results - KITTI - METER XXS

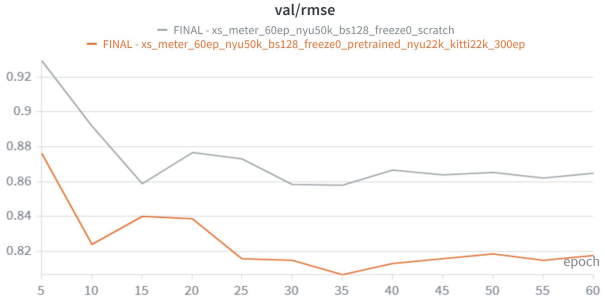


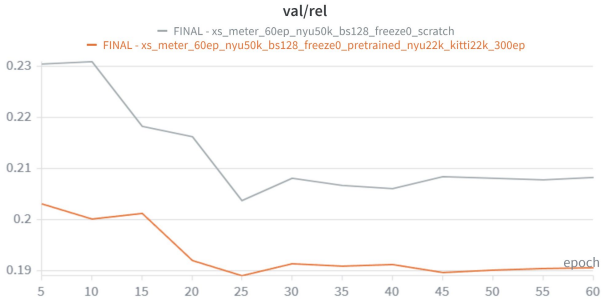


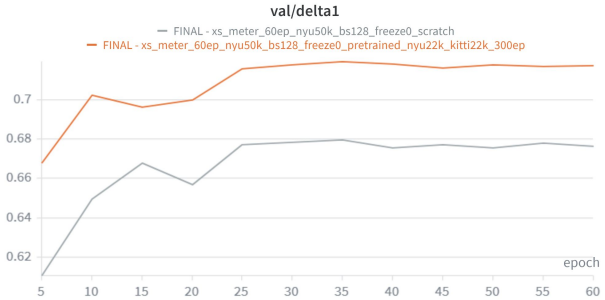


Metric	Supervised (Scratch)	LeJEPA Transfer (Ours)	Improvement
δ_1 (↑)	0.6187	**0.6825**	+10.31%
δ_2 (↑)	0.8666	**0.8893**	+2.62%
δ_3 (↑)	0.9460	**0.9521**	+0.64%
RMSE (↓)	0.9209	**0.8541**	-7.25%
AbsRel (↓)	0.2332	**0.2071**	-11.19%
log10 (↓)	0.1164	**0.1053**	-9.54%

Quantitative Results - NYU - METER XXS







Metric	Supervised (Scratch)	LeJEPA Transfer (Ours)	Improvement
δ_1 (↑)	0.6764	**0.7176**	+6.09%
δ_2 (↑)	0.8890	**0.9045**	+1.74%
δ_3 (↑)	0.9512	**0.9575**	+0.66%
RMSE (↓)	0.8650	**0.8177**	-5.47%
AbsRel (↓)	0.2082	**0.1906**	-8.45.%
log10 (↓)	0.1055	**0.0972**	-7.87%

Quantitative Results - NYU - METER XS

Conclusions & Future Work

Key findings:

- LeJEPA pretraining improves METER-XXS MDE by ~14% AbsRel / ~8% RMSE on average (NYU + KITTI)
- Mixed cross-domain pretraining (NYU+KITTI, 44k samples) outperforms single-domain pretraining on each individual dataset
- Full pipeline: ~12h on a consumer GPU (GTX 5060 Ti) - no cloud compute required
- Single hyperparameter (λ=0.02) - no teacher-student, no stop-gradient, easiness of implementation
- PCA of pretrained features shows spatially coherent representations capturing coarse scene geometry
- NYU XS variant experiments confirms the benefits of adding LeJEPA even to bigger models (+6–8% AbsRel)

Future work:

- Pretrain on ImageNet to test if general out-of-domain diversity benefits MDE
- Explore longer fine-tuning (>60 epochs) for pretrained models

END

Thanks for your attention!

References

- Balestriero, R., & LeCun, Y. (2025). LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics.
- Papa, L., Russo, P., & Amerini, I. (2024). METER: A Mobile Vision Transformer Architecture for Monocular Depth Estimation.
- Mur-Labadia, L., et al. (2026). V-JEPA 2.1: Unlocking Dense Features in Video Self-Supervised Learning.
- https://www.kaggle.com/datasets/qikangdeng/kitti-split-and-eigen-split
- https://www.cvlibs.net/datasets/kitti/eval_depth.php?benchmark=depth_prediction
- https://huggingface.co/datasets/sayakpaul/nyu_depth_v2
- https://www.youtube.com/watch?v=oM4neOyZOi0&feature=youtu.be

Appendix - Optimizations

- ***Problem:*** Loading full datasets into RAM causes OOM on resource-constrained machines; slow startup from decompressing archives every run
- ***Solution:*** **Memory-Mapped Data Loading**
  - One-time preprocessing converts raw data to pre-resized NumPy .npy files
  - At training time, the OS pages in only the samples needed per batch
  - RAM usage drops to near-zero regardless of dataset size

- ***Problem:*** Random initialization + ReLU causes the depth head to predict ~0, far from ground truth (NYU mean: 2.5m, KITTI mean: 11.5m). This leads to gradient starvation and training collapse
- ***Solution:*** **Dataset-Aware Depth Bias Initialization**
  - The final conv layer bias is initialized to a value close the dataset's median depth (we empirically choose 3.0 for NYU, 10.0 for KITTI)
  - "warm start" so predictions begin near ground truth, enabling healthy gradient flow from epoch 1